
Appendix

A Experiment Setup and Implementation Details

A.1 Fine-tuning Details

Base model. For reproducibility, we explicitly list the pretrained code foundation models used for fine-tuning on each dataset with link. On the OriGen dataset, we fine-tune DeepSeek-Coder-7B-Instruct-v1.5³ and CodeLlama-7B-Instruct⁴. On the Evol-CodeAlpaca-V1 dataset, we fine-tune DeepSeek-Coder-7B-v1.5⁵ and Meta-Llama-3-8B⁶.

Fine-tuning setup for FT. For the baseline FT, we employ the LoRA (Low-Rank Adaptation) method (Hu et al., 2022). Across all datasets and models, we adopt the AdamW optimizer with $\beta_1 = 0.9$ and $\beta_2 = 0.999$, cosine learning rate decay, and a warm-up ratio of 0.03. Training is performed for 3 epochs with a batch size of 4, using mixed `float16` precision (while the base models were pretrained in `bfloat16`). The LoRA configuration is fixed to $r = 32$ and a dropout rate of 0.05, while α varies by dataset. On the OriGen dataset, training is conducted on two NVIDIA A100 GPUs with 80 GB memory, using an initial learning rate of 5×10^{-5} and LoRA $\alpha = 32$. On the Evol-CodeAlpaca-V1 dataset, training is conducted on four NVIDIA H100 GPUs, with an initial learning rate of 1×10^{-4} and LoRA $\alpha = 64$.

Fine-tuning setup for IB-FT. Our proposed IB-FT method is also built on the LoRA (Low-Rank Adaptation) framework. Across all datasets, we adopt the AdamW optimizer with $\beta_1 = 0.9$ and $\beta_2 = 0.999$, cosine learning rate decay, and a warm-up ratio of 0.03. Training is performed for 3 epochs with a batch size of 4, using mixed `float16` precision (while the base models were pretrained in `bfloat16`). For LoRA configuration, we set the rank $r = 32$ and the dropout rate to 0.05, while α varies depending on the dataset. On the OriGen dataset, IB-FT is trained on two NVIDIA A100 GPUs with 80 GB memory, using an initial learning rate of 5×10^{-5} and LoRA $\alpha = 32$. On the Evol-CodeAlpaca-V1 dataset, IB-FT is trained on NVIDIA H100 GPUs, with an initial learning rate of 1×10^{-4} and LoRA $\alpha = 64$.

In addition to the above configurations, IB-FT introduces two extra hyperparameters in Eq. (IB-FT): the IB regularization strengths α and β , which balance the trade-off between compression and prediction. We search over $\alpha \in (0.01, 1)$ and $\beta \in (0.001, 0.1)$. On the OriGen dataset, we set $\alpha = 0.1$ and $\beta = 0.02$ for both models, while on the Evol-CodeAlpaca-V1 dataset we use $\alpha = 0.1$ and $\beta = 0.01$.

A.2 Evaluation Metrics

In addition to the standard $\text{Pass}@k$ metric, we adopt a stricter variant $\text{Pass}@k^{(m)}$ (as introduced in Fig. 2). Under $\text{Pass}@k^{(m)}$, a problem is counted as solved only if at least m out of the k generated samples pass the unit tests (so $\text{Pass}@k^{(1)}$ equals the conventional $\text{Pass}@k$). By requiring $m > 1$, $\text{Pass}@k^{(m)}$ penalizes lucky single-sample successes and provides a more reliable measure of a model’s consistent generation quality and robustness. To evaluate this metric, we need to generate a pool of n candidate outputs for each problem (with $n \geq k$). Among these n outputs, the parameters k and m are then defined relative to this pool: k is the subset size under evaluation, and m is the minimum number of successful outputs required within that subset. In our experiments, we fix $n = 10$.

B Additional Experiment Results

B.1 Performance of Fine-tuning on Evol-CodeAlpaca-V1

We also evaluate Llama-3-8B and DeepSeek-Coder-7B-v1.5 on Evol-CodeAlpaca-V1, comparing conventional fine-tuning (FT), IB-FT, and the base model before fine-tuning. Evaluation is carried out on the HumanEval, using both the standard $\text{Pass}@k$ metric ($k \in 1, 5, 10$) and the more demanding $\text{Pass}@10^{(m)}$ criterion (with $k = 10$, $m \in 2, 5, 10$).

As Table A1 shows, IB-FT consistently provides the strongest results across both models, with particularly notable improvements at $k = 1$. For instance, on Llama-3-8B, IB-FT achieves 61.60% for $\text{Pass}@1$ compared to 56.70% for FT, while on DeepSeek-Coder-7B-v1.5 the gains are 54.57% versus 51.28%. Moreover, under the strict $\text{Pass}@10^{(m)}$

³<https://huggingface.co/deepseek-ai/deepseek-coder-7b-instruct-v1.5>

⁴<https://huggingface.co/codellama/CodeLlama-7b-Instruct-hf>

⁵<https://huggingface.co/deepseek-ai/deepseek-coder-7b-base-v1.5>

⁶<https://huggingface.co/meta-llama/Meta-Llama-3-8B>

setting, IB-FT demonstrates substantially higher robustness, reaching 37.20% at $m = 10$ on Llama-3-8B, which still provides a clear improvement over FT’s 30.49%. These results highlight that IB-FT not only enhances top-1 accuracy but also yields more consistent and reproducible improvements under demanding multi-sample criteria, underscoring its advantage over conventional fine-tuning.

Table A1: Performance overview for Llama-3-8B and DeepSeek-Coder-7B-v1.5 fine-tuned on Evol-CodeAlpaca-V1 using IB-FT versus conventional fine-tuning (FT). Other metrics and table format follow those in Table 1.

Methods	HumanEval (%)					
	Pass@ k $\uparrow$			Pass@10 ^(m) $\uparrow$		
	$k = 1$	$k = 5$	$k = 10$	$m = 2$	$m = 5$	$m = 10$
Llama-3-8B						
Base	29.09	41.22	46.95	37.80	28.66	17.07
+ FT	56.70	74.16	79.27	73.17	62.80	30.49
+ IB-FT	61.60	75.44	79.88	73.78	65.24	37.20
DeepSeek-Coder-7B-v1.5						
Base	44.33	56.89	62.20	54.88	45.12	30.49
+ FT	51.28	64.10	64.10	63.46	50.00	32.05
+ IB-FT	54.57	67.30	70.73	67.07	54.88	39.63

B.2 Limitation of Pruning Memorized Examples, then Fine-tuning

Recall from Sec. 4 that removing highly-memorized examples (as characterized by $\text{Min-K\% Prob}$ scores) is a natural countermeasure to memorization. In Fig. A1 we further examine this strategy on two base models (DeepSeek-Coder-7B-Instruct-v1.5 and CodeLlama-7B-Instruct) by removing 10%–40% of the most-memorized examples and reporting Pass@ k for $k \in \{1, 5, 10\}$. The results reveal two sensitivity dimensions. First, the optimal removal ratio differs across models (*e.g.*, DeepSeek-Coder-7B-Instruct-v1.5 peaks at 10% for Pass@1 while CodeLlama-7B-Instruct peaks near 20%). Second, the optimal ratio varies with the metric even for the same model (*e.g.*, DeepSeek-Coder-7B-Instruct-v1.5’s Pass@1 peaks at 10% whereas Pass@5 peaks at 30%). As shown, there is no universal pruning rate: removal is model- and metric-dependent. By contrast, our proposal avoids fragile data pruning and can be compatible with memorized examples during fine-tuning.

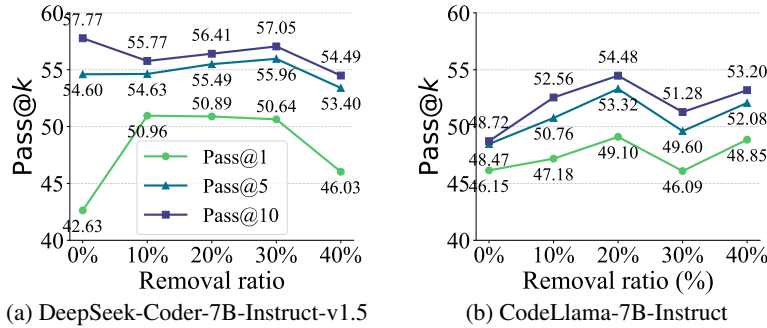


Figure A1: Performance of fine-tuning with different data-removal ratios on OriGen, evaluated on the VerilogEval (Eval-Human) subset. Fine-tuning is applied to the base model (a) DeepSeek-Coder-7B-Instruct-v1.5 and (b) CodeLlama-7B-Instruct, and the performance is assessed using Pass@ k ($k \in \{1, 5, 10\}$).